

Spec: PawMatch 雙軌制寵物社交 App 核心功能

1. 概述 (Overview)

本系統為一款「雙軌制寵物社交/交友 App」，旨在以毛孩為媒介，媒合「有寵物的使用者 (OWNER)」與「沒寵物的使用者 (LOVER)」。

- **技術棧限制**：前端 React Native (TypeScript)、後端 NestJS (TypeScript)、資料庫 PostgreSQL (搭配 Prisma ORM + PostGIS 擴充)、快取 Redis (ioredis)、環境配置 Docker Compose。

2. 需求規範 (Requirements)

2.1 雙軌制身份與認證 (Auth & Role-Based Access)

- **REQ-001**: 系統 SHALL 支持使用者使用 Email 與密碼進行註冊與登入。密碼必須經由 bcrypt 加密後方可存入資料庫。
- **REQ-002**: 使用者註冊時 SHALL 強制選擇身份：OWNER（有寵物）或 LOVER（沒寵物）。此身份一經註冊不得更改。
- **REQ-003**: 系統 SHALL 使用 JWT（JSON Web Token）進行 API 請求的身份驗證。
- **REQ-004**: 後端 NestJS SHALL 實作 RolesGuard。只有當 JWT Token 內身份為 OWNER 時，才允許呼叫 POST /pets（新增寵物）API；若為 LOVER 呼叫，系統 SHALL 回傳 403 Forbidden。

2.2 基於 PostGIS 的地理空間查詢 (Geolocation & Discovery)

- **REQ-005**: 前端 React Native SHALL 在 App 啟動時請求 GPS 定位權限，並透過邏輯進行「距離防抖」：只有當使用者移動超過 500 公尺時，前端才 SHALL 呼叫後端 API 更新最新經緯度。
- **REQ-006**: 後端 API GET /discover SHALL 接受 radius（公里數，預設 5）與分頁參數。
- **REQ-007**: 後端 NestJS SHALL 使用 Prisma 配合原生 SQL 查詢（prisma.\$queryRaw），利用 PostgreSQL 的 **PostGIS** 功能（如 ST_DWithin），高效篩選出當前使用者附近指定公里內的其他寵物檔案或 LOVER 用戶。

2.3 基於 Redis 的高性能滑卡匹配 (Swipe & Instant Match)

- **REQ-008:** 前端 React Native SHALL 提供流暢的卡片滑動介面 (Swipe Deck)，並支援點擊卡片左右側來切換瀏覽毛孩的相簿 (相簿至多 5 張照片)。
- **REQ-009:** 當用戶執行右滑 (LIKE) 時，前端 SHALL 呼叫後端 POST /swipes API。
- **REQ-010:** 後端 NestJS 收到 LIKE 請求時，SHALL 先將此互動紀錄寫入 **Redis Set 結構** (Key 格式: `user:${userId}:likes`)，**不應**即時寫入 PostgreSQL 以維持高併發效能。
- **REQ-011:** 寫入 Redis 後，後端 SHALL 即時使用 Redis 的 SISMEMBER 檢查被滑卡的使用者是否也曾喜歡過當前用戶。
- **REQ-012:** 若雙方互為 LIKE (即時匹配成功)，後端 SHALL 將此配對紀錄寫入 PostgreSQL Match 表，並即時回傳給前端 `{ matched: true, matchId: "xxx" }`；若未成功匹配，則回傳 `{ matched: false }`。

3. 行為情境 (Scenarios / Acceptance Criteria)

情境一：沒養寵物的使用者試圖繞過前端非法建立寵物檔案

- **Given** 使用者帳號註冊時的 role 為 LOVER
- **When** 該使用者繞過前端 UI，直接使用 Postman 向後端發送 POST /pets 請求
- **Then** NestJS 的 RolesGuard 應攔截該請求，並回傳 HTTP 狀態碼 403 Forbidden

情境二：使用者移動低於防抖門檻，不觸發後端效能消耗

- **Given** 使用者目前已登入 React Native 前端，且先前的 GPS 位置為 (A)
- **When** 使用者在客廳走動，GPS 偵測到新位置 (B)，且 A 與 B 的直線距離僅有 10 公尺
- **Then** 前端防抖邏輯應攔截此變更，不呼叫後端 API，直到移動距離超過 500 公尺

情境三：兩位飼主透過 Redis 實現即時配對成功

- **Given** 使用者小明 (ID: 1) 先前已右滑喜歡了使用者小美 (ID: 2)，此紀錄已存在於 Redis `user:1:likes` 中
- **When** 使用者小美 (ID: 2) 在前端也向右滑喜歡了小明 (ID: 1)，觸發後端 POST /swipes API
- **Then** 後端經由 Redis SISMEMBER 在 1 毫秒內發現小明也喜歡小美
- **And** 後端在 PostgreSQL 建立一筆 Match 紀錄
- **And** 後端回傳前端 `{ matched: true }`，前端 React Native 即時彈出「Match 成功」之精美互動 Modal